

普及组初赛模拟题（十二）

一、单项选择题（共15题，每题2分，共计30分；每题有且仅有一个正确选项）

1. 字符 D 的 ASCII 码为 68，则字符 Q 的 ASCII 码是：
 - A. 81
 - B. 82
 - C. 83
 - D. 视具体的计算机而定
2. 下列无符号数中，最小的数是：
 - A. $(1001100)_2$
 - B. $(114)_8$
 - C. $(75)_{10}$
 - D. $(4C)_{16}$
3. 线性表若采用链表存储结构，要求内存中可用存储单元地址（ ）。
 - A. 连续不连续均可
 - B. 必须连续
 - C. 部分地址必须连续
 - D. 一定不连续
4. 一棵结点数为 2021 的二叉树最多有（ ）个叶子结点。
 - A. 2020
 - B. 1011
 - C. 1010
 - D. 505
5. 如果 a 和 b 都是整型， a 的值为 5， b 的值为 2，那么 a/b 运算结果为：
 - A. 2
 - B. 2.5
 - C. 5
 - D. 1
6. 「后进先出」的数据结构是（ ）。
 - A. 链表
 - B. 队列
 - C. 邻接表
 - D. 栈

7. 小于 28 的正整数中，与 28 互质的数的个数为 ()。

- A. 10
- B. 11
- C. 12
- D. 13

8. 在 8 位二进制补码中，10110110 表示的是十进制下的 ()

- A. 202
- B. 74
- C. -202
- D. -74

9. 一个袋子中有 3 个蓝球，2 个红球，2 个黄球，则从中抽出三个球颜色各不相同的概率是多少?()

- A. $\frac{10}{21}$
- B. $\frac{13}{33}$
- C. $\frac{12}{35}$
- D. $\frac{3}{7}$

10. 定义 L 数:素数或者是回文数满足两者中任意一个条件的数。大于等于 10 并且小于等于 120 的“ L 数”共有多少个? (注:回文数指从左到右读与从右到左读是相同的, 如“121”、“1331”;两个条件都成立也是 L 数, 如“131”)()

- A. 34
- B. 35
- C. 36
- D. 37

11. 下面是根据欧几里得算法编写的函数，它所计算的是 a 和 b 的 ()。

```
1 | int euclid(int a, int b){
2 |     if (b == 0)
3 |         return a;
4 |     else
5 |         return euclid(b, a % b);
6 | }
```

- A.最大公共质因子
- B.最小公共质因子
- C.最大公约数
- D.最小公倍数

12. 在一个图中，所有顶点的度数之和等于所有边数的()倍。

- A. 2
- B. 4
- C. 1
- D. $\frac{1}{2}$

13. 某数列有 1000 个各不相同的数，由低到高按序排列，现要对该数列进行二分法检索，在最坏的情况下，需要检索()个数据。
- A. 1000
 - B. 10
 - C. 100
 - D. 500
14. 已知元素 (8, 25, 14, 87, 51, 90, 6, 19, 20)，问这些元素以怎样的顺序进入栈，才能使出栈的顺序满足：8 在 51 前面，90 在 87 的后面；20 在 14 的后面；25 在 6 的前面；19 在 90 的后面。（）
- A. 6, 25, 51, 8, 20, 19, 90, 87, 14
 - B. 19, 20, 90, 7, 6, 25, 51, 14, 87
 - C. 51, 6, 19, 20, 14, 8, 87, 90, 25
 - D. 20, 6, 8, 51, 90, 25, 14, 19, 87
15. 设某有向图的邻接表中有 n 个表头结点和 m 个表结点，则图中有多少条有向边（）。
- A. n
 - B. $n - 1$
 - C. m
 - D. $m - 1$

二、阅读程序（程序输入不超过数组或字符串定义的范围；判断题正确填 T，错误填 F；除特殊说明外，判断题 1.5 分，选择题 3 分，共计 40 分）

注意：判断题正确填 T，错误填 F。

阅读下面程序，完成第 16 ~ 21 题。

```

1  #include<bits/stdc++.h>
2  using namespace std;
3  int p;
4  void fun(int & x, int & y);
5  void func(int & x, int & y) {
6      if (y > x) return;
7      x--;
8      y /= 2;
9      fun(x, y);
10 }
11 void fun(int & x, int & y) {
12     if (x == 1) return;
13     x /= 2;
14     y += p;
15     func(x, y);
16 }
17 int main() {
18     int x, y;
19     cin >> x >> y >> p;
20     fun(x, y);
21     cout << x << ' ' << y;
22     return 0;
23 }

```

16. 将 void fun(int & x, int & y) 的 & 去除后,程序仍能通过编译。[]

17. 读入的 x, y, p 为 int 范围内任意值时程序均能完成运行。[]

18. 若 $x = 1$ 时,输出的 x, y 与输入的一致。[]

19. (2分) 输出的 x 必然小于等于输入的 x 。[]

20. 输入为 7 33 2 时,输出为()

- A. 4 31
- B. 3 35
- C. 3 31
- D. 4 35

21. 输入为 33 7 2 时,输出为()

- A. 5 3
- B. 3 5
- C. 6 4
- D. 4 6

阅读下面程序,完成第 22 ~ 27 题。

```

1  #include <stdio>
2  #define N 1005
3
4  using namespace std;
5  int num[N];
6
7  int main() {
8      int a1 = 1, n, x;
9      scanf("%d", &n);
10     num[1] = 1;
11     for (int i = 1; i <= n; ++i) {
12         x = 0;
13         for (int j = 1; j <= a1; ++j) {
14             num[j] = num[j] * 5 + x;
15             x = num[j] / 10;
16             num[j] %= 10;
17         }
18         if (x > 0) num[++a1] = x;
19     }
20     printf("0.");
21     for (int i = a1; i < n; ++i) {
22         putchar('0');
23     }
24     for (int i = a1; i >= 1; i--) {
25         printf("%d", num[i]);
26     }
27     putchar('\n');
28     return 0;
29 }

```

22. 程序输出的是 5^n 的值。[]

23. 程序执行到第 27 行 `putchar('\n');` 时, i 的值为 1。[]

24. 对于任意 $1 \leq i \leq a1$, 都有 $0 \leq num_i \leq 9$ 。[]

25. 程序输出的是一个小数, 且小数末尾可能有多余的 0。[]

26. 此程序的时间复杂度是 ()。

- A. $O(n \log n)$
- B. $O(n^2)$
- C. $O(n^3)$
- D. $O(n)$

27. (4分) 若 $n = 3$, 则输出为 ()。

- A. 0.125
- B. 0.8
- C. 8
- D. 125

阅读下面程序，完成第 28 ~ 33 题。

```
1  #include<iostream>
2
3  #include<cstring>
4
5  #include<cstdio>
6
7  #define N 500 + 10
8  using namespace std;
9  int a[N], n;
10 int main() {
11     cin >> n;
12     for (int i = 1; i <= n; i++) cin >> a[i];
13     for (int i = 1; i < n; i++) {
14         int tmp = i;
15         for (int j = i + 1; j <= n; j++)
16             if (a[j] < a[tmp]) tmp = j;
17         swap(a[i], a[tmp]);
18     }
19     for (int i = 1; i <= n; i++) cout << a[i] << " ";
20     cout << endl;
21     return 0;
22 }
```

28. 上述代码实现了对一个长度为 n 的序列进行排序[]。
29. 去掉头文件 `#include<cstdio>` 后程序仍能正常编译运行[]。
30. 去掉 `using namespace std;` 后程序仍能正常编译运行[]。
31. 我们将上述算法称为 ()
- A. *KMP*
 - B. 冒泡排序
 - C. 选择排序
 - D. 归并排序
32. 上述代码的时间复杂度为 ()
- A. $O(n)$
 - B. $O(n\log n)$
 - C. $O(n * n)$
 - D. $O(n * n * n)$
33. (4分) 若输入数据为：

5
3 2 1 5 4

则 `if(a[j] < a[tmp])` 这句话中的条件会成立多少次（即后面的 `tmp = j` 会被执行多少次）（

- A. 5
- B. 7
- C. 3
- D. 4

三、完善程序（单选题，每小题3分，共计30分）

阅读下面题目，完成第 34 ~ 38 题。

小 C 到商店购物，她只带了 SW 元钱。有 n 件商品，第 i 件商品的价格为 w_i 元，小 C 对它的满意度为 v_i 。小 C 想要知道，用自己仅有的 SW 元钱，能买到的所有商品的满意度之和最大是多少。数据保证 $1 \leq SW \leq 10000$ 。

```
1  #include <iostream>
2  using namespace std;
3
4  const int INF = 0x3f3f3f3f;
5
6  int n, SW, W[105], V[105], Dp[①];
7
8  int main()
9  {
10     cin >> n >> SW;
11     for (int i = 1; i <= n; i++)
12         cin >> W[i] >> V[i];
13     for (int i = 1; i <= SW; i++)
14         Dp[i] = ②;
15     for (int i = 1; i <= n; i++)
16         for (③)
17             Dp[j] = max(Dp[j], ④);
18     int ans = 0;
19     for (int i = 1; i <= SW; i++)
20         ⑤;
21     cout << ans << "\n";
22     return 0;
23 }
```

34. ①处应填（）

- A. 100
- B. 105
- C. 10000
- D. 10005

35. ②处应填 ()

- A. INF
- B. 0
- C. -1
- D. 1

36. ③处应填 ()

- A. `int j = SW; j >= W[i]; j--`
- B. `int j = W[i]; j <= SW; j++`
- C. `int j = 1; j <= n; j++`
- D. `int j = n; j >= 1; j--`

37. ④处应填 ()

- A. `Dp[i - W[j]] + V[j]`
- B. `Dp[i - V[j]] + W[j]`
- C. `Dp[j - W[i]] + V[i]`
- D. `Dp[j - V[i]] + W[i]`

38. ⑤处应填 ()

- A. `ans = max(ans, Dp[i])`
- B. `ans = min(ans, Dp[i])`
- C. `ans = ans + Dp[i]`
- D. `ans = ans - Dp[i]`

阅读下面题目，完成第 39 ~ 43 题。

给出一堆长度各异的木棍，这些木棍能否头尾相连形成一个正方形？

输入：

第一行包含一个整数 N ，为数据组数。接下来 N 行，每行第一个数 M 代表木棍的数量，第 2 至 $M + 1$ 个数为每个木棍的长度。

输出：

对每组数据，输出 yes 或 no 表示这组木棍是否能拼成正方形。

样例输入

```
3
4 1 1 1 1
5 10 20 30 40 50
8 1 7 2 6 4 4 3 5
```

样例输出

yes
no
yes

```

1  #include<iostream>
2
3  #include<algorithm>
4
5  #include<cstdio>
6
7  #include<cstring>
8
9  using namespace std;
10
11  const int MAXN = 25;
12  int side;
13  int m;
14  int sticks[MAXN];
15  bool visit[MAXN];
16
17  bool DFS(int sum, int number, int position) {
18      if ( ____(1)___ ) {
19          return true;
20      }
21      int sample = 0;
22      for (int i = position; i < m; ++i) {
23          if (visit[i] || sum + sticks[i] > side || ____(2)___ ) {
24              continue;
25          }
26          visit[i] = true;
27          if ( ____(3)___ ) {
28              if (DFS(0, number + 1, 0)) {
29                  return true;
30              } else {
31                  sample = sticks[i];
32              }
33          } else {
34              if (DFS(sum + sticks[i], number, i + 1)) {
35                  return true;
36              } else {
37                  sample = sticks[i];
38              }
39          }
40          visit[i] = false;
41      }
42      return false;
43  }
44
45  bool compares(int x, int y) {
46      return x > y;
47  }
48
49  int main() {
50      int n;

```

```

51     scanf("%d", &n);
52     while (n--) {
53         int length = 0;
54         scanf("%d", &m);
55         for (int i = 0; i < m; i++) {
56             scanf("%d", &sticks[i]);
57             length += sticks[i];
58         }
59         memset(visit, false, sizeof(visit));
60         if (length % 4 != 0) {
61             printf("no\n");
62             continue;
63         }
64         side = length / 4;
65         sort(sticks, sticks + m, Compare);
66         if ( ____ (4) ____ ) {
67             printf("no\n");
68             continue;
69         }
70         if ( ____ (5) ____ ) {
71             printf("yes\n");
72         } else {
73             printf("no\n");
74         }
75     }
76     return 0;
77 }

```

39. (1) 处应填 () 。

- A. number == 0
- B. number == 1
- C. number == 2
- D. number == 3

40. (2) 处应填 () 。

- A. sticks[i] < sample
- B. sticks[i] == sample
- C. sticks[i] > sample
- D. sticks[i] != sample

41. (3) 处应填 () 。

- A. sum + sticks[i] < side
- B. sum + sticks[i] == side
- C. sum + sticks[i] > side
- D. sum + sticks[i] != side

42. (4) 处应填 () 。

- A. sticks[0] < side

- B. `sticks[0] == side`
- C. `sticks[0] > side`
- D. `sticks[0] != side`

43. (5) 处应填 () 。

- A. `DFS(0, 0, 1)`
- B. `DFS(0, 0, m)`
- C. `DFS(0, 0, 0)`
- D. `DFS(length/4, 0, 0)`